Multivariate analysis exam

UCT Statistics Honours 2026

Table of contents

Questions

Analysis I

```
if (!exists("read_exam_data", mode = "function")) {
  stop("Please run the first chunk to define the 'read_exam_data' function.")
}
data_tidy_stress <- read_exam_data("data_tidy_stress.csv")
```

```
Rows: 500 Columns: 6
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (6): age, income, satisfaction, experience, hours, stress
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Question 1.1

Part a

```r
# View(data_tidy_stress)
# Calculate correlation matrix:
cor_mat = cor(data_tidy_stress)
cor_mat
```

```
                   age     income satisfaction experience       hours
age          1.00000000 0.83271764    0.6392036 0.96823135  0.06373114
income       0.83271764 1.00000000    0.7284207 0.80566405  0.06219594
satisfaction 0.63920359 0.72842066    1.0000000 0.62688896 -0.16060620
experience   0.96823135 0.80566405    0.6268890 1.00000000  0.06068512
hours        0.06373114 0.06219594   -0.1606062 0.06068512  1.00000000
stress       0.02257214 0.04061594   -0.2540621 0.01528969  0.73491605
                  stress
age           0.02257214
income        0.04061594
satisfaction -0.25406213
experience    0.01528969
hours         0.73491605
stress        1.00000000
```

Part b

```r
# PCA approach:
# Eigen decmposition:
eig_pca = eigen(cor_mat)

# First 2 eigenvalues:
eig_pca_val = as.matrix(eig_pca$values)
val = eig_pca_val[1:2]

# First 2 eigenvectors:
eig_pca_vec = as.matrix(eig_pca$vectors)
vec = as.matrix(eig_pca_vec[,1:2])
```

```
# loadings = sqrt(eigenvalues)*eigenvectors:
l = vec%*%diag(sqrt(val))
l = as.data.frame(l)
colnames(l) = c("Factor 1","Factor 2")
l
```

```
       Factor 1    Factor 2
1 -0.9512598432  0.09766994
2 -0.9254431943  0.08412459
3 -0.8151629908 -0.25191938
4 -0.9407003437  0.09272898
5 -0.0009138101  0.92179470
6  0.0574696456  0.93056422
```

Part c

```
# Obtain loadings using ML approach:
fa_mod = factanal(covmat=cor_mat,factors=2,rotation='none')
print(fa_mod$loadings,cutoff=0)
```

```
Loadings:
             Factor1 Factor2
age           0.993   0.044
income        0.837   0.058
satisfaction  0.659  -0.241
experience    0.973   0.036
hours         0.032   0.738
stress       -0.021   0.997

                Factor1 Factor2
SS loadings       3.069   1.603
Proportion Var    0.512   0.267
Cumulative Var    0.512   0.779
```

Part d

The loadings obtained from the ML approach are more interpretable than the loadings obtained from the PC approach. Age, Income, Satisfaction and experience have large positive loadings on factor 1. Factor 1 represents an individuals work expertise. Hours and Stress have large

positive loadings onto factor 2, while the other variables have very weak loadings. Factor 2 explains the physical demands related to work. As both stress and average hours worked per week increase, factor 2 increases.

Part e

Factor Analysis aims at identifying unobserved variables that explain the covariance structure in the observed data. If a high-variance variable, that is uncorrelated with the other variables, is added the the dataset the ML loadings will be less effected than the PCA loadings as the ML approach regards the variable specific variance and error variance as unexplained. The PCA loadings will The PCA approach aims at reproducing the covariance structure of X, since the new variable is uncorrelated there will be minimal change in the loadings but they will all increase slightly.

Part f

Yes, factor scores must be uncorrelated. The points in Figure 1 are randomly scattered suggesting that there is no correlation between factor 1 and factor 2.

Analysis II

```
if (!exists("read_exam_data", mode = "function")) {
  stop("Please run the first chunk to define the 'read_exam_data' function.")
}
img_list <- read_exam_data("pca_image_results.rds")
```

Question 2.1

Part a

No. PCA aims to create linear combinations of the explanatory variables in a manner that maximises variance. PCA must be performed on standardised data or variables with high variance will dominate the first principal component. Variables with large variance are more informative than variables with less variance but individual variable variation cannot be properly accounted for if all variables are not measured on the same scale.

Part b

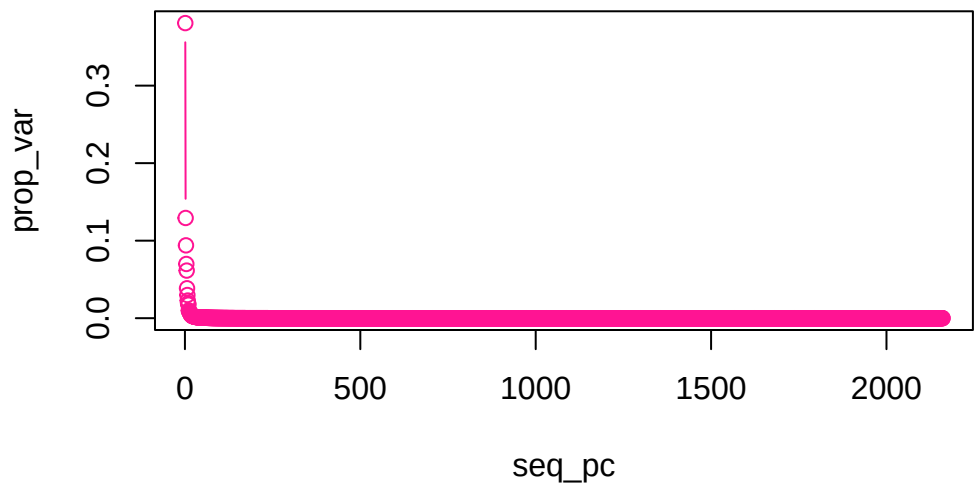
```
# View(img_list)
# Reconstruct original image:
scores      = img_list$Y
loadings    = img_list$V
eigenvalues = img_list$lambda
means       = img_list$M

og_img = t(t(scores%*%t(loadings))+means)
save_img(og_img,'reconstructed_image.jpg')
```

```
[1] "reconstructed_image.jpg"
```

```
knitr::include_graphics('reconstructed_image.jpg')
```

Part c

```r
# Determine optimal no. principal components:
# Total variance:
total_var = sum(eigenvalues)

# Proportion of variance explained:
prop_var = eigenvalues/total_var

# Principal components:
seq_pc = seq(1:length(eigenvalues))

# Plot proportion of variance explained by components:
plot(seq_pc,prop_var,
     type='b',
     col='deeppink')
```



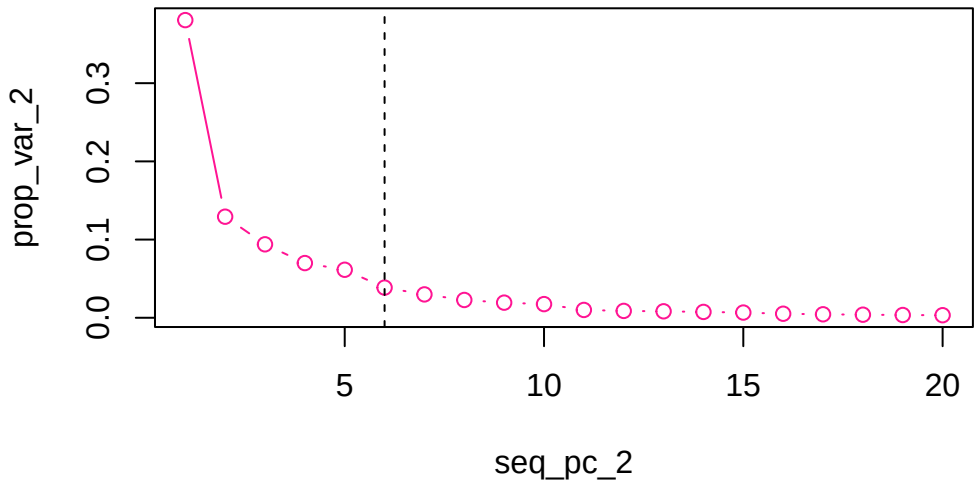
The above plot is difficult to interpret since there are such a large number of principal components. The following plot was produced using only the first 20 principal components.

```r
# Proportion of variance explained:
# First 20 PCs:
prop_var_2 = prop_var[1:20]
seq_pc_2 = seq(1:20)
```

```r
cum_var_2 = cumsum(prop_var_2)
cum_var_2
```

```
 [1] 0.3806797 0.5099566 0.6039041 0.6738627 0.7352764 0.7738711 0.8037712
 [8] 0.8265442 0.8458789 0.8633410 0.8733514 0.8821211 0.8904048 0.8978988
[15] 0.9045720 0.9097204 0.9141751 0.9180983 0.9216237 0.9249002
```

```r
# Plot proportion of variance explained by first 20 PCs:
plot(seq_pc_2,prop_var_2,
     type='b',
     col='deeppink')
abline(v=6,lty=2)
```



Based on the above elbow plot, the optimal number of principal components is 6. Including additional components does not result in large increase in the proportion of variation explained. The first 6 principal components explain approximately 77% of the variation in the data.

```r
# Reconstruct image:
# Scores:
scores_2 = as.matrix(img_list$Y)[1:6,1:2160]

# First 6 loading vectors:
```

```r
loadings_2 = as.matrix(img_list$V)[1:2160,1:2160]

# Means
means_2 = img_list$M[1:2160]

recon_img = t(t(scores_2%*%t(loadings_2))+means_2)
save_img(recon_img,'reconstructed_image_2.jpg')
```

```
[1] "reconstructed_image_2.jpg"
```

```r
knitr::include_graphics('reconstructed_image_2.jpg')
```

Clearly, 6 principal components dont' accurately reconstruct the image.

Analysis III

```r
if (!exists("read_exam_data", mode = "function")) {
  stop("Please run the first chunk to define the 'read_exam_data' function.")
}
data_tidy_lum_response <- read_exam_data("data_tidy_lum_response.csv")
```

```
Rows: 100 Columns: 16
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): pid
dbl (15): crp, ddimer, fractalkine, ifng, il10, il1a, il1ra, mcp1, mip1a, mi...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
data_tidy_lum_meta <- read_exam_data("data_tidy_lum_meta.csv")
```

```
Rows: 100 Columns: 6
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (6): pid, infxn, age, sex, ethnicity, household_size

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
data_tidy_lum <- data_tidy_lum_meta |>
  dplyr::left_join(data_tidy_lum_response, by = "pid") |>
  dplyr::mutate(
    age_infxn = paste0(age, "_", infxn)
  ) |>
  dplyr::select(pid:household_size, age_infxn, everything())
```

Question 3.1

Part a

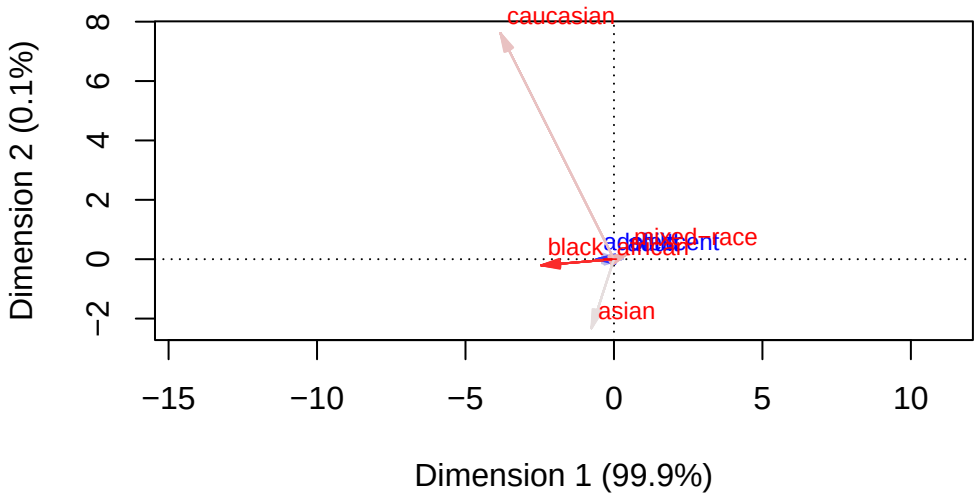
```
library(ca)
# View(data_tidy_lum_meta)
# Create a row-principal plot:
# row = age
# col = ethnicity

# dat = cbind(data_tidy_lum_meta$age,data_tidy_lum_meta$ethnicity)
# dat = table(dat)

# Frequency table:
X = matrix(c(1,2,9,30,38,20,2,3,4,0,0,1),
           nrow=3,
           ncol=4)
X = as.data.frame(X)
colnames(X) = c("black_african","mixed-race","asian","caucasian")
rownames(X) = c("child","adult","adolescent")
X
```

```
           black_african mixed-race asian caucasian
child                  1         30     2         0
adult                  2         38     3         0
adolescent             9         20     4         1
```

```r
# Correspondence matrix:
P = X/sum(X)
P
```

```
           black_african mixed-race      asian   caucasian
child        0.009090909  0.2727273 0.01818182 0.000000000
adult        0.018181818  0.3454545 0.02727273 0.000000000
adolescent   0.081818182  0.1818182 0.03636364 0.009090909
```

```r
# CA:
ca_obj = ca(P)

# Plot row-principal plot:
plot(ca_obj,
     map='rowprincipal',
     mass=c(TRUE,TRUE),
     contrib='absolute',
     arrows=c(TRUE,TRUE))
```



Part b

Dimension 1 is explaining approximately all the data and is splitting mixed race from black african, caucasian and asian individuals. Mixed race lies at the origin suggesting that the distribution of individuals follows the expected distribution under the assumption of independence. Since there is only one individual who is classified as Caucasian the corresponding arrow is very long, it is an extreme value.

Part c

The chi-squared distance is a weighted Euclidean distance. It is calculated by subtracting the expected value from the observed value, squaring the answer and dividing by the expected value. Euclidean distance is the square of the distance between the observed andf expected values. It does not take into account the mass profiles of the columns/rows. The chi-squared distance is preferred as it downweights abundant columns (or rows) resulting in a more accurate representation of distances. If a paerticular row has a large column mass the distances between observations may appear large in comparison to another row with a smaller column mass. Large distances when there are a lot of observations are less informative than large distances when there are only a few observations. The chi-squared distance takes this distribution of obsrrvatins into account.

Part d

PCA aims to create linear combinations of variables to maximise variance. PCA would yield a component where mixed_race loads very strongly onto the first component as this is a much more abundant column. PCA would not capture the variation in the other ethnicities as they would be dominated by mixed race.

Question 3.2

Part a

```
# Construct X and Y matrices:
n = nrow(data_tidy_lum_response)
X = cbind(1,data_tidy_lum_response$ifng,data_tidy_lum_response$il10)
Y = cbind(data_tidy_lum_response$crp,data_tidy_lum_response$ddimer)

beta_hat = solve(t(X)%*%X)%*%t(X)%*%Y
beta_hatdf = as.data.frame(beta_hat)
rownames(beta_hatdf) = c("intercept","crp","ddimer")
colnames(beta_hatdf) = c("ifng","il10")
beta_hatdf
```

```
                ifng        il10
intercept  1.17243385  0.93615170
crp       -0.25001550  0.13990375
ddimer    -0.04736831 -0.03495714
```

Part b

```
# Full model fitted above
Yhat_full = X%*%beta_hat
res_full = Y-Yhat_full
MLE_full = (t(res_full)%*%(res_full))/n
MLE_full1 = MLE_full[1,1] # ifng
MLE_full2 = MLE_full[2,2] # il10

# Restricted model
Xres = cbind(1,data_tidy_lum_response$il10)
Yres = cbind(data_tidy_lum_response$crp,data_tidy_lum_response$ddimer)
beta_hat_res = solve(t(Xres)%*%Xres)%*%t(Xres)%*%Yres

Yhat_res = Xres%*%beta_hat_res
res_res = Yres-Yhat_res
MLE_res = (t(res_res)%*%res_res)/n
MLE_res1 = MLE_res[1,1]
MLE_res2 = MLE_res[2,2]

# Parameters:
n = nrow(data_tidy_lum_response)
k = ncol(X)
r = ncol(Y)
p = k-1
q = ncol(X)-ncol(Xres)

# Test statistic:
lambda1 = (MLE_full1/MLE_res1)^(n/2)
lambda2 = (MLE_full2/MLE_res2)^(n/2)

stat1 = -(n-k-0.5*(r-p+q+1))*(lambda1^(2/n))
stat2 = -(n-k-0.5*(r-p+q+1))*(lambda2^(2/n))

chi1 = pchisq(stat1,df=r*q)
chi2 = pchisq(stat2,df=r*q)
```

```r
ans = as.data.frame(c(chi1,chi2))
rownames(ans) = c("ifng","il10")
ans
```

```
     c(chi1, chi2)
ifng             0
il10             0
```

The p-value = 0 suggests that ifng has a significant effect on both both crp and dimmer.

Part c

Part d

```r
dat = data_tidy_lum_response[,-(1:2)]

# Correlation matrix:
X = cor(dat)

# Eigen decomposition:
eigX = eigen(X)
eigX_val = as.matrix(eigX$values)
eigX_vec = as.matrix(eigX$vectors)

# Principal components:
pc = as.matrix(t(t(eigX_vec[,1:2])%*%X))
pc
```

```
                [,1]        [,2]
ddimer       0.34733988  0.40175663
fractalkine -0.64174788  0.71328010
ifng         0.34347177 -0.14218349
il10        -0.55982082 -0.38199308
il1a         0.20417109 -0.04278215
il1ra       -0.17198702 -0.13445941
mcp1        -0.44413427 -0.08478640
mip1a        0.18385464 -0.17530143
mip1b       -0.44839492 -0.63581479
mmp9         0.82343087 -0.46469190
```

```
sap          -0.05295570  0.65089499
tnfa         -0.34331925 -0.42980346
vegf         -0.04748255  0.30261226
scd40l        0.37918848  0.34862453
```

```r
Y_pcr = as.matrix(data_tidy_lum_response[,2])
X_pcr = cbind(1,pc)

# beta_pcr = solve(t(X_pcr)%*%X_pcr)%*%t(X_pcr)%*%Y_pcr
```

Question 3.3

In `data_tidy_lum_response`, the variables `crp` to `il1ra` are one set of variables ($\mathbf{X}^{(1)}$), and the variables `mcp1` to `scd40l` are another set ($\mathbf{X}^{(2)}$).

Standardise the variables in `data_tidy_lum_response`, creating a new matrix `lum_response_mat_std`, using this code, and use `lum_response_mat_std` for question 3.3:

```r
if (!exists("data_tidy_lum")) {
  stop("Please run code above to create the `data_tidy_lum` object.")
}
lum_response_mat_std <- data_tidy_lum |>
  dplyr::select(crp:scd40l) |>
  dplyr::mutate(across(dplyr::everything(), ~ (.-mean(.))/sd(.))) |>
  as.matrix()
```

Part a

```r
# Create matrices:
X1 = as.matrix(lum_response_mat_std[,1:7])
X2 = as.matrix(lum_response_mat_std[,8:15])

# Fit model:
cca_mod = cancor(X1,X2)

#
cor1 = cca_mod$cor[1]
a1 = cca_mod$xcoef[,1,drop=FALSE]
b1 = cca_mod$ycoef[,1,drop=FALSE]
round(cor1,3)
```

```
[1] 0.519
```

```
round(a1,3)
```

```
              [,1]
crp         -0.002
ddimer      -0.004
fractalkine  0.093
ifng         0.005
il10         0.025
il1a         0.012
il1ra       -0.054
```

```
round(b1,3)
```

```
         [,1]
mcp1   -0.015
mip1a   0.030
mip1b  -0.002
mmp9   -0.093
sap     0.019
tnfa   -0.022
vegf    0.020
scd40l -0.015
```

```
U1 = X1%*%a1
V1 = X2%*%b1
round(head(U1),3)
```

```
       [,1]
[1,] -0.043
[2,]  0.165
[3,] -0.061
[4,]  0.137
[5,] -0.106
[6,] -0.066
```

```
round(head(V1),3)
```

```
       [,1]
[1,]  0.029
[2,]  0.195
[3,]  0.070
[4,]  0.076
[5,]  0.099
[6,] -0.003
```

U1 and V1 have a correlation of approximately 0.52. fracalkine is has the largest absolute value in the a coefficients and thus loads most strongly onto the first canonical variable,U1. mmp9 has the largest absolute value in the b coefficients and thus loads most strongly into the first canonical variable, V1.

Part b

```
a4 = cca_mod$xcoef[,4,drop=FALSE]
b4 = cca_mod$ycoef[,4,drop=FALSE]
# round(a4,3)
# round(b4,3)

cor4 = cca_mod$cor[4]
round(cor4,3)
```

```
[1] 0.292
```

```
U4 = X1%*%a4
V4 = X2%*%b4
round(head(U4),3)
```

```
       [,1]
[1,]  0.076
[2,]  0.141
[3,] -0.013
[4,] -0.040
[5,] -0.143
[6,]  0.143
```

```
round(head(V4),3)
```

```
       [,1]
[1,]  0.112
[2,]  0.154
[3,]  0.001
[4,] -0.146
[5,] -0.181
[6,]  0.030
```

Part c

```
helpp = lm(V4~X1)
summary(helpp)
```

```
Call:
lm(formula = V4 ~ X1)

Residuals:
      Min        1Q    Median        3Q       Max
-0.185126 -0.062689 -0.001442  0.054808  0.282342

Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)   -6.990e-18  9.970e-03   0.000    1.000
X1crp          1.568e-03  1.030e-02   0.152    0.879
X1ddimer       6.138e-03  1.012e-02   0.607    0.546
X1fractalkine  6.387e-03  1.038e-02   0.615    0.540
X1ifng         8.190e-05  1.069e-02   0.008    0.994
X1il10        -2.609e-02  1.030e-02  -2.534    0.013 *
X1il1a        -1.167e-02  1.034e-02  -1.129    0.262
X1il1ra        5.877e-05  1.029e-02   0.006    0.995
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.0997 on 92 degrees of freedom
Multiple R-squared:  0.08549,   Adjusted R-squared:  0.01591
F-statistic: 1.229 on 7 and 92 DF,  p-value: 0.2953
```

```
helpp_hat = helpp$fitted.values
round(cor(helpp_hat,V4),3)
```

```
      [,1]
[1,] 0.292
```

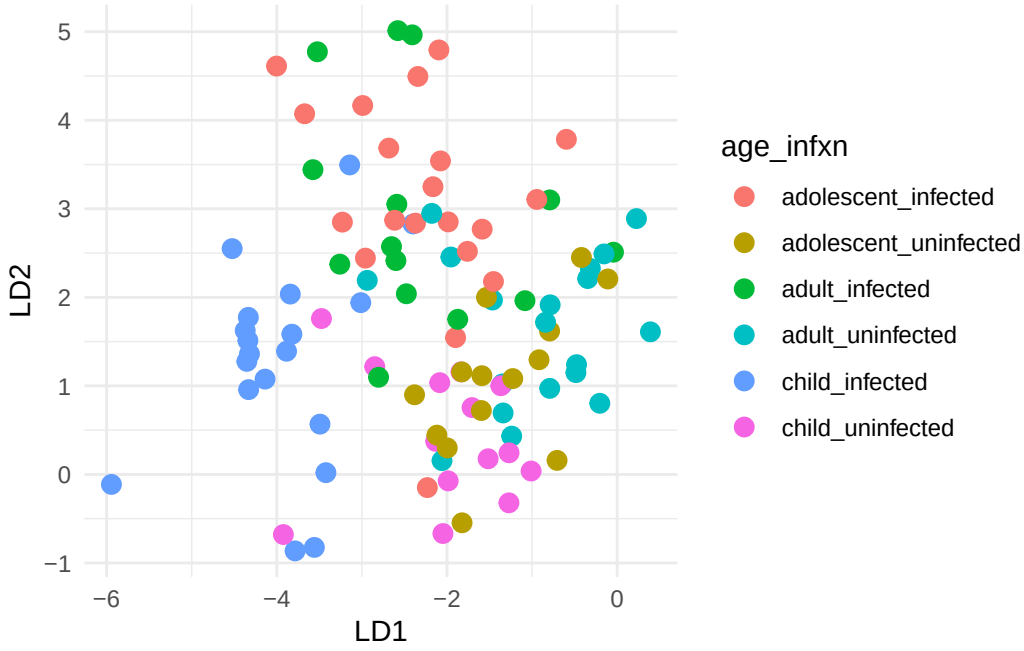
The correlation is the exact same (0.292).

Question 3.4

Part a

```
# View(data_tidy_lum)
# Step 1:
dat = data_tidy_lum[,-(1:6)]
# Make groups numeric:
# I know there is a more efficient way to do this, I just forgot :(
# dat$age_infxn_num = ifelse(dat$age_infxn=='child_uninfected',1,
  #                          ifelse(dat$age_infxn=='child_infected',2,
    #                               ifelse(dat$age_infxn=='adult_uninfected',3,
      #                                    ifelse(dat$age_infxn=='adult_infected',4,
        #                                        ifelse(dat$age_infxn=='adolescent_infected

X = dat[,-1] # remove age_infxn
# X = dat[,-16] # remove age_infxn_num
g = length(unique(dat$age_infxn)) # no. groups

n_i = table(dat$age_infxn)
n_ii = c("adolescent_infected","adolescent_uninfected","adult_infected","adult_uninfected","

bar_overall = colMeans(X)
bar_group = dat|>
  dplyr::group_by(age_infxn)|>
  dplyr::summarise(
    across(everything(),mean),
    .groups='drop'
  )|>
  dplyr::select(-age_infxn)|>
  as.matrix()

#v Step 2:
B = Reduce(`+`,lapply(1:g,function(i){
  n_i[i]*((bar_group[i,]-bar_overall)%*%t(bar_group[i,]-bar_overall))
}))
```

```r
W = Reduce(`+`,lapply(1:g,function(i){
  Xi = X[dat$age_infxn==n_ii[i],]
  cov(Xi)*(n_i[i]-1)
}))

# Step 3:
eig = eigen(solve(W)%*%B)
eig_vec =eig$vectors |> Re()
S_pooled = W/(sum(n_i)-g)
sd_vec = diag(1/S_pooled)|>sqrt()
eig_vec = (eig_vec*sd_vec) |>as.matrix()
eig_vec = eig_vec[,1:2]

# Step 4:
X = as.matrix(X)
scores = X%*%eig_vec
dat$LD1 = scores[,1]
dat$LD2 = scores[,2]

# Step 5:
ggplot(data=dat,
       aes(x=LD1,y=LD2,color=age_infxn))+
  geom_point(size=3)+
  theme_minimal()
```



Part b

```
l_tbl = as.data.frame(eig_vec)
colnames(l_tbl) = c("LD1","LD2")
rownames(l_tbl) = c("V1","V2","V3","V4","V5","V6","v7","V8","V9","V10","V11","V12","V13","V14
rownames(l_tbl) = colnames(X)
round(l_tbl,3)
```

```
               LD1    LD2
crp         -0.006 -0.022
ddimer      -0.797  1.086
fractalkine -0.522 -0.511
ifng        -0.330  0.494
il10        -0.077  0.086
il1a        -0.242 -0.107
il1ra        0.160 -0.132
mcp1        -0.405 -0.034
mip1a        0.052  0.055
mip1b        0.051  0.049
mmp9         0.784  0.710
sap          0.104 -0.049
tnfa        -0.186  0.126
```

```
vegf         -0.609 -0.047
scd40l        0.231  0.317
```

In the above plot, LD1 is distinguishing between infected children and everyone else. LD2 is distinguishing between infected adolescents and uninfected children. There is still a lot of overlap between the 6 groups, which suggests that 2 discriminants are not sufficient to accurately separate the age_infxn groups. LD1 is driven by ddimmer (-1.159), fracalkine (-0.654), ifng (-0.695) and mcp1 (-0.631). All of these variables have strong, negative loadings into LD1. LD2 is driven by ddimer (1.579), ifng (1.042), with the remaining variables loading weakly on to LD2. ddimer and ifng load negatively onto LD1 but positively onto LD2. This suggests that both ddimer and ifng are strong indicators of age_infxn.

Multiple choice

- 4.1: a
- 4.2: d
- 4.3: c
- 4.4: b
- 4.5: a
- 4.6: a
- 4.7:
- 4.8: